
**An-Najah National University**

Faculty of Engineering and Information Technology

Department of Computer Engineering


Software Graduation Project

# MedOrbit

Presented in partial fulfillment of the requirements for
Bachelor degree in Computer Engineering


**Group Members:**

Ali Ghassan Abdullah Dawood

Omar Abdullah Sadeq Abu Mazen

**Supervisor:**

Dr. Bashar Tahyana


2026

# Acknowledgment

We would like to express our sincere gratitude to our supervisor, Dr. Bashar Tahyana, for his guidance, technical feedback, and continuous support throughout the development of MedOrbit. His observations helped us improve the project scope, software architecture, implementation decisions, and the way the system was evaluated. We would also like to thank the Department of Computer Engineering at An-Najah National University for providing the academic background that made this project possible. We are grateful to our families and friends for their encouragement and patience during the development and testing period. Finally, we would like to thank each other as a team for the cooperation, problem solving, and continuous effort required to integrate the web, mobile, backend, database, artificial intelligence, and deployment parts of the project into one system.

# Disclaimer

This report was written in fulfillment of the requirements for a Bachelor degree in Computer Engineering. It has not been altered or corrected, other than editorial corrections, as a result of assessment. The views, results, and recommendations presented in this report are based on the work completed by the project team. MedOrbit is an academic software prototype. Its artificial-intelligence features are designed to provide informational assistance and software demonstrations and are not a substitute for professional diagnosis, treatment, emergency services, or the judgment of a licensed healthcare professional. The project is not presented as a certified medical device or as a production healthcare system compliant with every national or international healthcare regulation.

# Abstract

MedOrbit is a smart healthcare platform developed to integrate a wide range of healthcare services into one digital environment for users in Nablus, Palestine. The system provides both a web application and a Flutter mobile application connected to a shared Node.js/Express backend and a PostgreSQL database with PostGIS support. The platform is designed around multiple user roles, including patients, doctors, administrators, and super administrators, with role-specific interfaces and authorization rules. The implemented platform supports account registration and authentication, Google Sign-In, doctor discovery, clinic and healthcare-facility discovery on a map, appointment scheduling, medical records, electronic prescriptions, notifications, patient–doctor relationships, direct messaging, doctor applications, social content, administrative management, audit-related functions, and subscription/billing workflows. Arabic and English user experiences are supported, including right-to-left presentation where required. A separate Python/FastAPI artificial-intelligence service extends the platform with medical question answering, symptom triage and specialty recommendation, drug-interaction checking, medical-report summarization, and a Virtual Doctor workflow. The AI subsystem uses natural-language processing, retrieval-augmented generation, a locally hosted large language model through Ollama, OCR/document extraction for medical reports, and explicit safety and identity boundaries between the public clients and the internal AI service. The system is organized as a multi-service architecture. PostgreSQL/PostGIS stores application and geospatial data, while Apache Kafka and background workers support transactional-outbox events and recommendation processing. Docker Compose provides a reproducible local environment for the backend, frontend, database, AI service, Kafka, and background workers. Automated testing and continuous-integration definitions cover authentication, authorization, clinical access boundaries, administration, doctor lifecycle, messaging, recommendations, billing, database migration, frontend contracts, and AI safety-related behavior. The final implementation demonstrates how healthcare management, telemedicine-oriented workflows, intelligent assistance, location-based discovery, communication, and role-based administration can be combined in a single software platform. MedOrbit is intended as an academic prototype and decision-support environment rather than a replacement for professional medical care.

# Nomenclature and Abbreviations

| Abbreviation | Meaning |
|---|---|
| AI | Artificial Intelligence |
| API | Application Programming Interface |
| CORS | Cross-Origin Resource Sharing |
| CI | Continuous Integration |
| DB | Database |
| EHR/EMR | Electronic Health Record / Electronic Medical Record |
| HTTP | Hypertext Transfer Protocol |
| JWT | JSON Web Token |
| LLM | Large Language Model |
| NLP | Natural Language Processing |
| NLU | Natural Language Understanding |
| OCR | Optical Character Recognition |
| RAG | Retrieval-Augmented Generation |
| RBAC | Role-Based Access Control |
| REST | Representational State Transfer |
| RTL | Right-to-Left user-interface direction |
| SQL | Structured Query Language |
| TTS | Text-to-Speech |
| STT | Speech-to-Text |
| UI | User Interface |
| UX | User Experience |
| WebSocket | Full-duplex communication protocol used for real-time features |

# Contents

# List of Figures

# List of Tables

# Chapter 1

# Introduction

## 1.1 Statement of the Problem

Healthcare information and services are often distributed across different channels. A patient may use one method to search for a doctor, another method to contact a clinic, a separate process to arrange an appointment, and paper or isolated digital records for prescriptions and medical history. This fragmentation makes it harder for users to move smoothly between discovery, communication, scheduling, and personal healthcare information. A second problem is access to structured guidance. Users may know their symptoms but not the appropriate medical specialty, may need a quick way to review possible interactions between medications, or may receive medical reports that are difficult to understand. General-purpose online search can provide large amounts of information, but it does not organize the user's healthcare workflow or connect the result to doctors, clinics, appointments, and personal records in one place. Communication also becomes difficult when patient, doctor, and administrative functions are implemented as unrelated systems. Doctors need controlled access to patient information and scheduling tools, while administrators need mechanisms for reviewing doctor applications, managing users, moderating public content, and auditing sensitive actions. These needs require clear authorization boundaries rather than one universal interface. MedOrbit addresses these problems by providing a unified healthcare platform. The same backend serves web and mobile clients, while role-based authorization determines which operations are available to each user. The platform combines traditional healthcare-management functions with location-based discovery, direct communication, and AI-assisted services.

## 1.2 Objectives

The main objective of MedOrbit is to design and implement a unified, secure, and usable smart-healthcare platform that connects patients, doctors, healthcare facilities, administrative services, and intelligent assistance. The project aims to:

- Develop web and mobile applications that use a shared backend and database.
- Support secure registration, login, verification, password recovery, and Google Sign-In.
- Implement separate patient, doctor, administrator, and super-administrator experiences.
- Allow users to discover doctors by specialty and locate clinics and healthcare facilities in Nablus.
- Provide appointment scheduling and doctor self-service scheduling workflows.
- Digitize medical records and electronic prescriptions with controlled doctor/patient access.
- Support patient–doctor communication through direct messaging and notifications.
- Provide doctor-application and administrative-review workflows.
- Integrate AI-assisted symptom triage and medical-specialty recommendation.
- Provide a drug-interaction checker and prescription-related warnings.
- Extract and summarize medical-report content in Arabic and English.
- Provide an AI medical chatbot and a structured Virtual Doctor workflow.
- Support multilingual Arabic/English presentation and RTL interfaces.
- Use event-driven processing for selected notifications, signals, and recommendations.
- Package the main services in Docker to improve reproducibility and deployment consistency.
- Apply automated testing to important authentication, authorization, clinical, administrative, billing, and AI behaviors.

## 1.3 Significance

The significance of MedOrbit comes from integrating several computer engineering and software-engineering areas in one project. The system combines full-stack development, mobile development, relational and geospatial databases, REST APIs, authentication and authorization, real-time communication, event-driven architecture, artificial intelligence, OCR, local large-language-model integration, containerization, and automated testing. From the user's perspective, the project reduces the need to move between unrelated tools. A patient can discover a doctor, inspect healthcare facilities, arrange

an appointment, communicate with a doctor, access records and prescriptions, and use intelligent assistance through the same platform. From the doctor's perspective, MedOrbit provides a dedicated workspace for professional information, schedules, appointments, patients, records, and communication. Administrators receive separate management and review capabilities. The project is also significant as an engineering exercise because medical software requires more careful boundaries than a normal social or e-commerce application. MedOrbit therefore treats AI output as assistance, keeps authenticated AI operations behind the backend boundary, distinguishes human doctor messaging from AI conversations, and applies role-aware access controls to sensitive operations.

## 1.4 Project Scope

MedOrbit is a graduation-project prototype focused on healthcare services for Nablus and on demonstrating an integrated architecture. The project scope includes:

- A responsive web interface implemented with HTML, CSS, and JavaScript and served through nginx.
- A Flutter mobile application with role-aware navigation.
- A Node.js/Express backend exposing REST endpoints and real-time messaging capabilities.
- PostgreSQL as the primary relational database and PostGIS for geospatial facility data.
- A Python/FastAPI AI service for chatbot, triage, drug checking, report summarization, and Virtual Doctor functions.
- Local LLM inference through Ollama, with retrieval and safety layers in the AI pipeline.
- Kafka-based event processing and background workers for outbox, event, and recommendation workflows.
- Docker-based development and integration environments, with staging deployment documentation.

The project does not claim to replace a hospital information system, certified EHR product, licensed telemedicine provider, pharmacist, or medical professional. Medical AI output must be interpreted as assistance rather than a definitive diagnosis. Payment functionality in the project also includes development/sandbox-oriented flows and should not be treated as a production financial system without provider, compliance, and security review.

## 1.5 Organization of the Report

This report is organized into six chapters. Chapter 1 introduces the problem, objectives, significance, and project scope. Chapter 2 presents the theoretical background and literature review related to digital health, telemedicine, electronic records, healthcare AI, RAG, geospatial discovery, and software security. Chapter 3 explains the methodology, system architecture, implementation technologies, database, web and mobile applications, AI service, event-driven components, and testing approach. Chapter 4 presents the implementation results and analyzes the delivered features. Chapter 5 discusses engineering challenges, security and medical safety considerations, limitations, and the relationship between the final system and the original objectives. Chapter 6 concludes the report and presents recommendations for future work.

# Chapter 2

# Theoretical Background and Literature Review

## 2.1 Digital Health Platforms

Digital health uses computing, communication, and data technologies to support healthcare delivery and health-system management. The World Health Organization emphasizes that digital-health initiatives require coordinated technical, organizational, human, and financial resources rather than a single isolated technology [1]. This perspective is relevant to MedOrbit because the project is not implemented as one feature; it joins identity, healthcare records, discovery, communication, administration, and AI services through a common architecture. A unified platform can reduce context switching and make related information available through controlled workflows. However, integration also increases the importance of security boundaries: when authentication, records, messaging, appointments, and AI share one backend, an authorization mistake can affect several features. For this reason, system integration must be combined with explicit access-control rules and testing.

## 2.2 Telemedicine and Remote Healthcare Communication

Telemedicine and digital communication can reduce geographic and scheduling barriers between patients and healthcare providers. In a software platform, remote healthcare is more than a video link. It depends on identity, appointment context, communication history, doctor availability, and access to the correct patient data. A useful design therefore connects remote communication with scheduling and records rather than treating it as an independent page. MedOrbit's architecture provides the surrounding services required for such workflows: doctor accounts, care relationships, appointment

records, direct messaging, notifications, and database support for video-consultation entities. This allows remote-care functions to be placed inside a broader patient–doctor relationship model.

## 2.3 Electronic Medical Records and Electronic Prescriptions

Electronic medical records provide a structured way to store healthcare information and make it available to authorized users. The central engineering issue is not only storage; it is ownership and access. A patient should be able to view appropriate personal information, while a doctor should only perform clinical operations when the relationship and role permit them. Electronic prescriptions have similar constraints. Prescription data must be associated with the correct doctor and patient and should preserve medication details in a structured form. MedOrbit extends this workflow by connecting prescription creation with AI-assisted drug-interaction checking. The AI result is treated as a warning layer, while the doctor remains responsible for clinical decisions.

## 2.4 Artificial Intelligence in Healthcare

AI can support healthcare information processing, but health-related AI requires strong attention to safety, transparency, privacy, and human oversight. WHO guidance highlights both the potential of AI in medicine and the ethical risks that can arise when systems are deployed without adequate governance [2]. These concerns are especially important for generative systems because fluent language can appear more certain than the underlying evidence. MedOrbit therefore separates the public clients from the AI service. The backend authenticates persisted AI operations and passes an internal identity context to the FastAPI service. The AI code also contains a medical safety layer, structured triage logic, and explicit feature-specific pipelines instead of sending every request directly to an unrestricted language model.

## 2.5 Symptom Triage and Specialty Recommendation

Symptom triage in MedOrbit is intended to guide a user toward an appropriate specialty and follow-up action; it is not designed to establish a medical diagnosis. The implemented AI service accepts a set of symptoms, applies a rule-based matching en-

gine, calculates specialty-related scores and a confidence value, and persists the resulting triage session. This structured approach provides a more controlled output than asking a generative model to make an unconstrained diagnosis.

## 2.6 Drug-Interaction Checking

Drug-interaction checking compares medications and returns known interaction information and severity. In MedOrbit, the AI service can accept medication names or identifiers, query database-backed interaction data, and produce a severity summary. A related prescription-check endpoint can generate warnings when a prescription contains interacting medications. This design illustrates an important software principle: deterministic or database-backed logic should be preferred when the problem can be represented with explicit data. The large language model is therefore not used as the sole authority for drug-interaction facts.

## 2.7 Medical Report Extraction and Summarization

Medical reports may arrive as text, PDFs, or scanned images. A summarization pipeline must first obtain usable text before a language model can summarize it. MedOrbit supports text input and file input. PDF/image input can pass through document extraction and OCR before a bilingual Arabic/English summary is produced. The processing result is stored together with metadata such as source type, processing time, and model information. The purpose of this feature is readability and organization, not diagnostic interpretation. Extracted text can contain OCR mistakes, and generated summaries can omit or misstate details. The original report should therefore remain the authoritative source.

## 2.8 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) combines information retrieval with a generative language model. Instead of relying only on the model's internal parameters, the system retrieves context from a controlled knowledge source and supplies that context during response generation. MedOrbit includes a RAG subsystem and uses a locally hosted LLM through Ollama for medical question-answering workflows. RAG can improve grounding, but it does not eliminate the need for safety controls. Retrieval may return incomplete context, and a language model can still produce an incorrect interpretation. For this reason, RAG is one part of the MedOrbit AI pipeline rather than a guarantee of medical correctness.

## 2.9 Geospatial Healthcare Discovery

Healthcare discovery requires both descriptive data and location data. MedOrbit stores facility information in PostgreSQL/PostGIS and presents facility discovery through web mapping and Flutter mapping components. Geospatial support allows the system to represent clinics and facilities relative to a user's location and to combine map-based browsing with doctor and specialty information.

## 2.10 Role-Based Access Control and Healthcare Privacy

Role-Based Access Control (RBAC) maps permissions to roles instead of allowing every authenticated user to perform the same operations. MedOrbit uses distinct patient, doctor, administrator, and super-administrator surfaces. Backend middleware and route-level policies enforce protected operations, while the clients adapt navigation according to the user's role. RBAC is necessary but not sufficient for healthcare data. Clinical access can also depend on the relationship between a particular doctor and patient. MedOrbit therefore includes care-relationship concepts in addition to broad role checks.

## 2.11 Event-Driven Architecture

An event-driven design is useful when a user-facing transaction should not wait for every secondary operation. MedOrbit includes a transactional outbox, Kafka, an event consumer, and a recommendation consumer. The transactional-outbox pattern stores an event as part of the database transaction and lets a worker publish it later. This reduces the risk of a database change succeeding while its related event is silently lost.

## 2.12 Literature Review and Gap Analysis

The reviewed concepts show several common categories of health software. Telemedicine systems emphasize communication, EHR systems emphasize records, doctor directories emphasize discovery, and AI assistants emphasize question answering or triage. A major design goal of MedOrbit is to combine these categories in one academic platform while still preserving clear service and authorization boundaries.

Table 2.1: Functional gap addressed by MedOrbit.

| **System Category** | **Typical Primary Focus** | **MedOrbit Integration** |
|---|---|---|
| Doctor directory | Search and profile discovery | Doctor discovery, specialties, clinics, maps, appointments |
| Telemedicine application | Remote communication | Appointments, doctor relationships, direct messaging, consultation-oriented data |
| Electronic record system | Structured clinical information | Medical records, prescriptions, doctor/patient authorization |
| AI health assistant | Conversational or analytical guidance | Chatbot, triage, drug checking, report summarization, Virtual Doctor |
| Administrative portal | Operational management | User management, doctor applications, moderation, audit logs, invitations |
| Recommendation system | Personalized content/discovery | Event-driven signals and recommendation processing integrated with the platform |

The project's contribution is therefore not a claim that every individual feature is new. Its contribution is the engineering integration of these features into one web/mobile healthcare prototype, using a shared backend, role-aware access, geospatial data, AI services, asynchronous events, and a reproducible containerized environment.

# Chapter 3

# Methodology and Experimental Methods

## 3.1 Overall Methodological Approach

MedOrbit was developed incrementally. The system was divided into independent but connected domains: authentication, doctor and clinic discovery, appointments, clinical records, communication, AI, administration, mobile parity, billing, and event-driven processing. Each domain was implemented and tested before being integrated into the larger platform. The practical development approach followed these stages:

1. Analyze the project requirements and define user roles.
2. Design the shared database and REST API boundaries.
3. Implement authentication and authorization foundations.
4. Build the web client and connect it to the backend.
5. Implement doctor, clinic, appointment, records, prescriptions, and communication features.
6. Build and expand the Flutter mobile client against the same backend contracts.
7. Implement the separate FastAPI AI service and integrate it through authenticated backend routes.
8. Introduce event-driven outbox and recommendation processing using Kafka.
9. Add administration, doctor application, audit, notification, and billing-related workflows.
10. Package the services with Docker Compose.
11. Add automated tests and CI gates for critical contracts.
12. Perform integration and physical-device validation, with final screenshots documented separately.

## 3.2 System Architecture

MedOrbit follows a service-oriented full-stack architecture. Users interact through either the web interface or Flutter mobile application. Both clients communicate with the Node.js/Express backend. The backend is the primary security and identity boundary for application operations. It accesses PostgreSQL/PostGIS, communicates with the internal FastAPI AI service, and produces or consumes asynchronous events where appropriate.

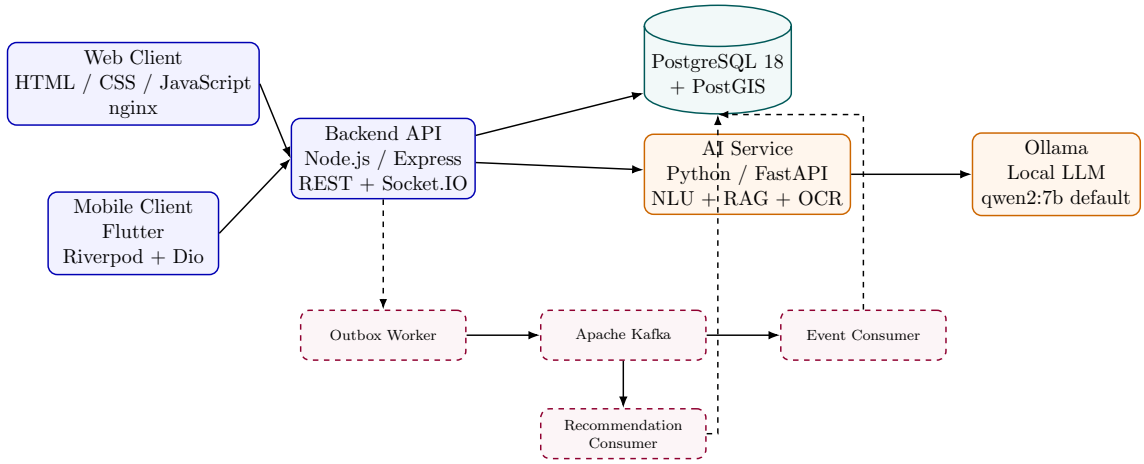



Figure 3.1: High-level architecture of the MedOrbit platform.

## 3.3 Technology Stack

Table 3.1: Main MedOrbit implementation technologies.

| Layer | Technology | Purpose |
|---|---|---|
| Web frontend | HTML, CSS, JavaScript, nginx | Responsive browser UI and static delivery |
| Mobile | Flutter / Dart | Android-oriented cross-platform mobile application |
| Mobile state/navigation | Riverpod, GoRouter, Dio | State management, routing, HTTP communication |
| Backend | Node.js, Express | REST API, authorization, business logic |
| Real-time communication | Socket.IO | Direct messaging and real-time events |
| Database | PostgreSQL 18 | Transactional application data |
| Geospatial extension | PostGIS | Clinic/facility location and geographic queries |
| AI API | Python, FastAPI | AI feature orchestration and internal API |
| AI/NLP | NLU, RAG, local LLM via Ollama | Medical assistance and grounded Q&A |
| Document processing | pdfplumber, pypdf, Tesseract OCR | Report text extraction |
| Voice | faster-whisper, Piper TTS | Voice-oriented Virtual Doctor capabilities |
| Messaging/event platform | Apache Kafka | Asynchronous event and recommendation pipeline |
| Deployment | Docker / Docker Compose | Reproducible multi-service environment |
| Testing/CI | GitHub Actions, Node/Python tests, Flutter tests | Automated verification |

## 3.4 Repository Structure

The implementation is organized into separate top-level components:

- `backend/`: Express API, repositories, services, middleware, routes, workers, migrations, and integration tests.
- `frontend/`: static web pages, JavaScript/CSS modules, nginx configuration, and frontend tests.
- `mobile/`: Flutter application organized into core, features, routes, and shared components.

- `ai-service/`: FastAPI AI implementation including chatbot, NLU, RAG, medical tools, Virtual Doctor, and tests.
- `docs/`: API-contract and staging/deployment documentation.
- `docker-compose.yml`: local integrated service topology.

## 3.5 Backend Design

The backend uses Express and is divided into configuration, routes, controllers, repositories, services, middleware, events, utilities, and workers. This separation reduces the amount of business logic placed directly in route handlers and makes authorization and data access easier to test. The application exposes route groups for authentication, users, doctors, clinics, appointments, medical records, prescriptions, patients, feedback, reviews, reports, AI operations, Virtual Doctor, billing, notifications, specialties, social feed, recommendations, conversations, direct messages, contact messages, doctor applications, care relationships, administration, system settings, audit logs, and invitations.

### 3.5.1 Authentication

Authentication includes conventional account flows and Google Sign-In. JWT is used for authenticated API sessions, while password handling uses bcrypt-based hashing. Verification and password-recovery flows are separated from normal authenticated operations.

### 3.5.2 Authorization

Authorization is role aware. Public read operations are available for selected discovery data, while protected write or clinical operations pass through authentication and authorization middleware. Administrative functions are separated from doctor and patient operations, and super-administrator-only functions exist for sensitive tasks such as administrator invitations.

### 3.5.3 Security Middleware

The Express application applies several defensive controls:

- Helmet-generated security headers plus explicit security-policy headers.
- Configured CORS rules.
- API rate limiting and a stricter AI-chat fair-use limiter.
- Request-size limits.
- Centralized error handling and API 404 handling.

- Authentication boundaries before persisted AI operations.

These controls improve the prototype's security posture, but they do not by themselves establish formal healthcare-regulatory compliance.

## 3.6 Database Design

MedOrbit uses PostgreSQL with a dedicated `medorbit` schema and PostGIS for location-aware data. The database evolves through ordered SQL migrations. The migration history covers authentication foundations, authorization invalidation, administrator invitations, doctor applications, care relationships, social feed, direct messaging, video consultations, transactional outbox, recommendations, scheduling, and later platform features. The main conceptual groups include:

- users, sessions, verification and authentication state;
- doctors, specialties, clinics/facilities, and doctor applications;
- appointments, schedules, and consultation-related data;
- medical records and prescriptions;
- care relationships between patients and doctors;
- notifications, direct messages, chatbot conversations, and social content;
- AI triage/report-summarization data;
- billing/subscription data;
- audit, administration, and invitation records;
- event/outbox and recommendation-related data.

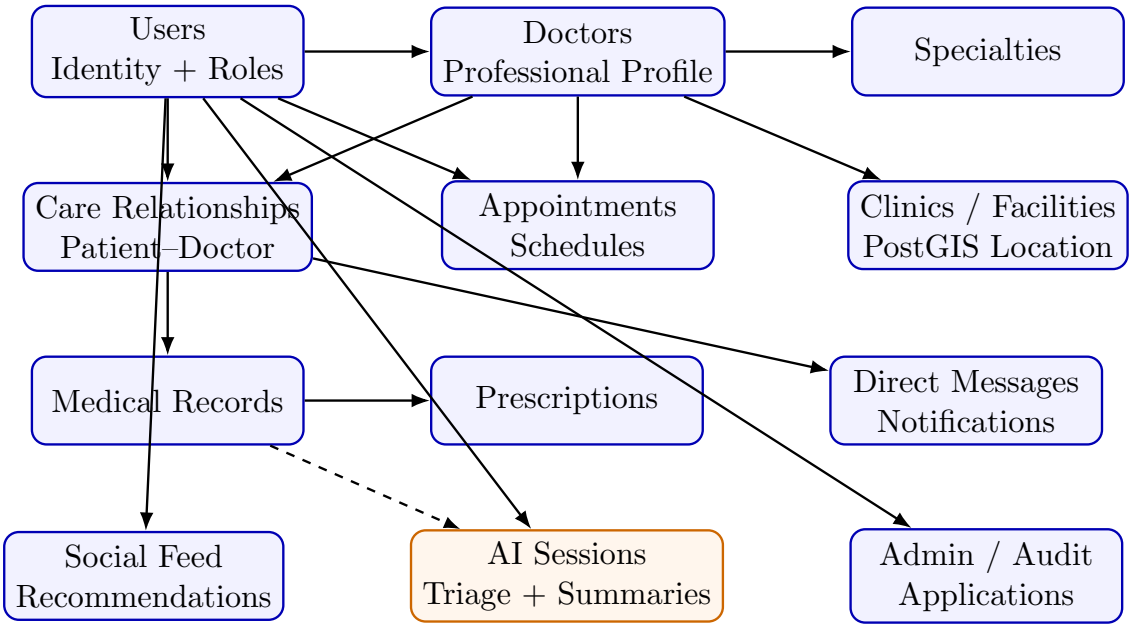



Figure 3.2: Simplified conceptual data model for the main MedOrbit domains.

Figure 3.2 is intentionally a conceptual view rather than a full physical schema. The

production database contains additional supporting tables for sessions, verification, migrations, events, billing, moderation, and other implementation details.

## 3.7 Web Application

The web client is implemented without a frontend framework. It uses HTML, CSS, and JavaScript and is served by nginx. The browser client communicates with the backend API and provides pages for public discovery, authentication, patients, doctors, AI tools, billing, communication, and administration. The use of a static web frontend keeps the browser build pipeline simple, while shared JavaScript modules handle authentication state, API calls, layout, localization, and feature-specific behavior.

### 3.7.1 Web Localization

MedOrbit supports Arabic and English. Language application is handled in shared JavaScript so direction and visible text can be updated consistently. RTL support is particularly important for Arabic healthcare content and form interaction.

## 3.8 Mobile Application

The mobile client is implemented using Flutter and Dart. Dio is used for HTTP communication, Riverpod for application state, GoRouter for navigation, Flutter Secure Storage for protected local token/state storage, and Google Sign-In for the Google authentication flow. Mapping and location functions use Flutter Map, geographic-coordinate packages, clustering, and geolocation libraries. Socket.IO client support is included for real-time communication. The route map demonstrates broad feature parity with the platform. It contains routes for authentication, feed, discovery, services, records, prescriptions, appointments, feedback, notifications, profile, symptom checker, drug checker, report summarizer, reports, doctors, shared notes, saved places, contact, doctor application, Virtual Doctor, maps, clinics, chatbot conversations, billing, direct messaging, doctor workspace, and administrative functions.

### 3.8.1 Patient Mobile Experience

The patient experience focuses on a small set of high-frequency tasks: discovering care, arranging appointments, reviewing personal healthcare data, contacting doctors, and accessing AI assistance. Navigation groups these functions so the user does not need to understand the internal service architecture.

### 3.8.2 Doctor Mobile Workspace

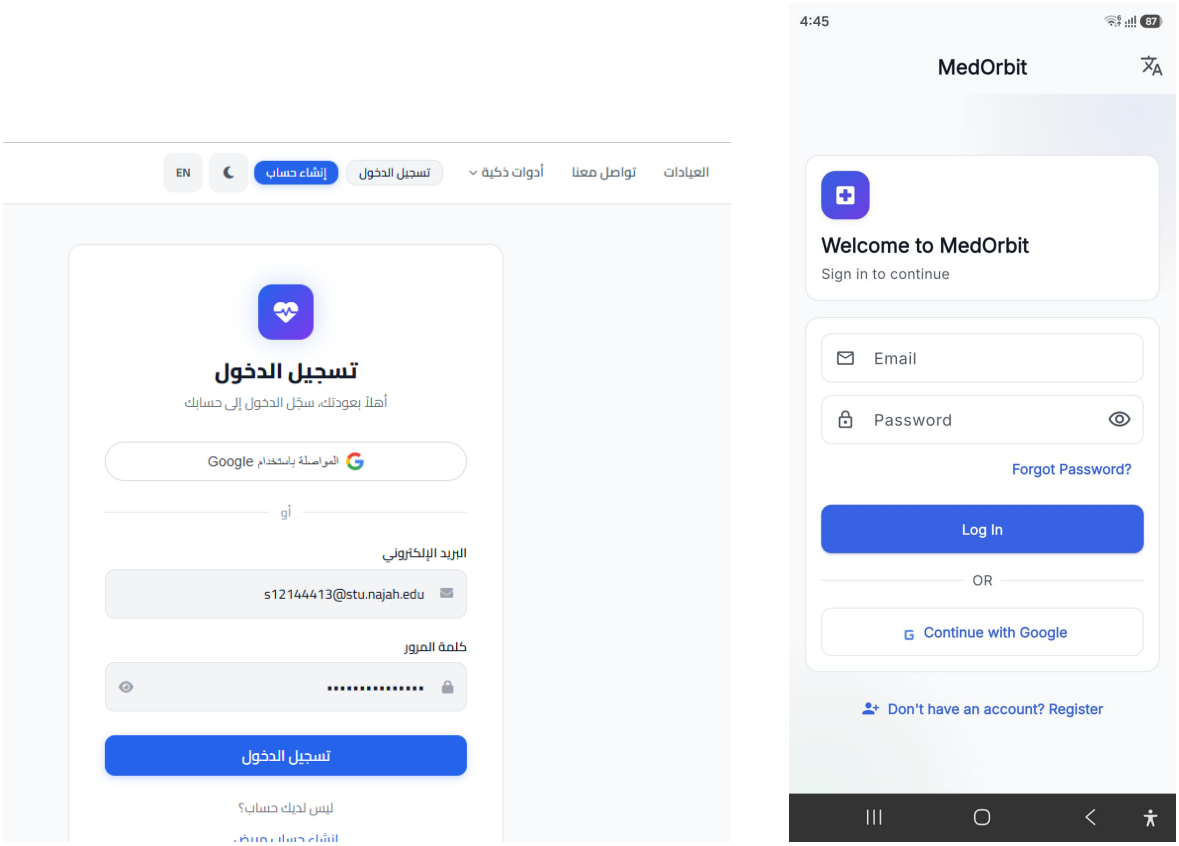
Doctors have dedicated routes for professional profile management, schedule, appointments, patient lists, patient detail, posts, and records. This separates professional workflows from the general patient interface and reduces the risk of exposing doctor-only actions in shared screens.

### 3.8.3 Administration on Mobile

Administrator and super-administrator routes include dashboard, analytics, user management, doctor-application review, contact messages, moderation, audit logs, and invitations. The invitation flow is intentionally separated because a person accepting an invitation may not yet have an administrator role.

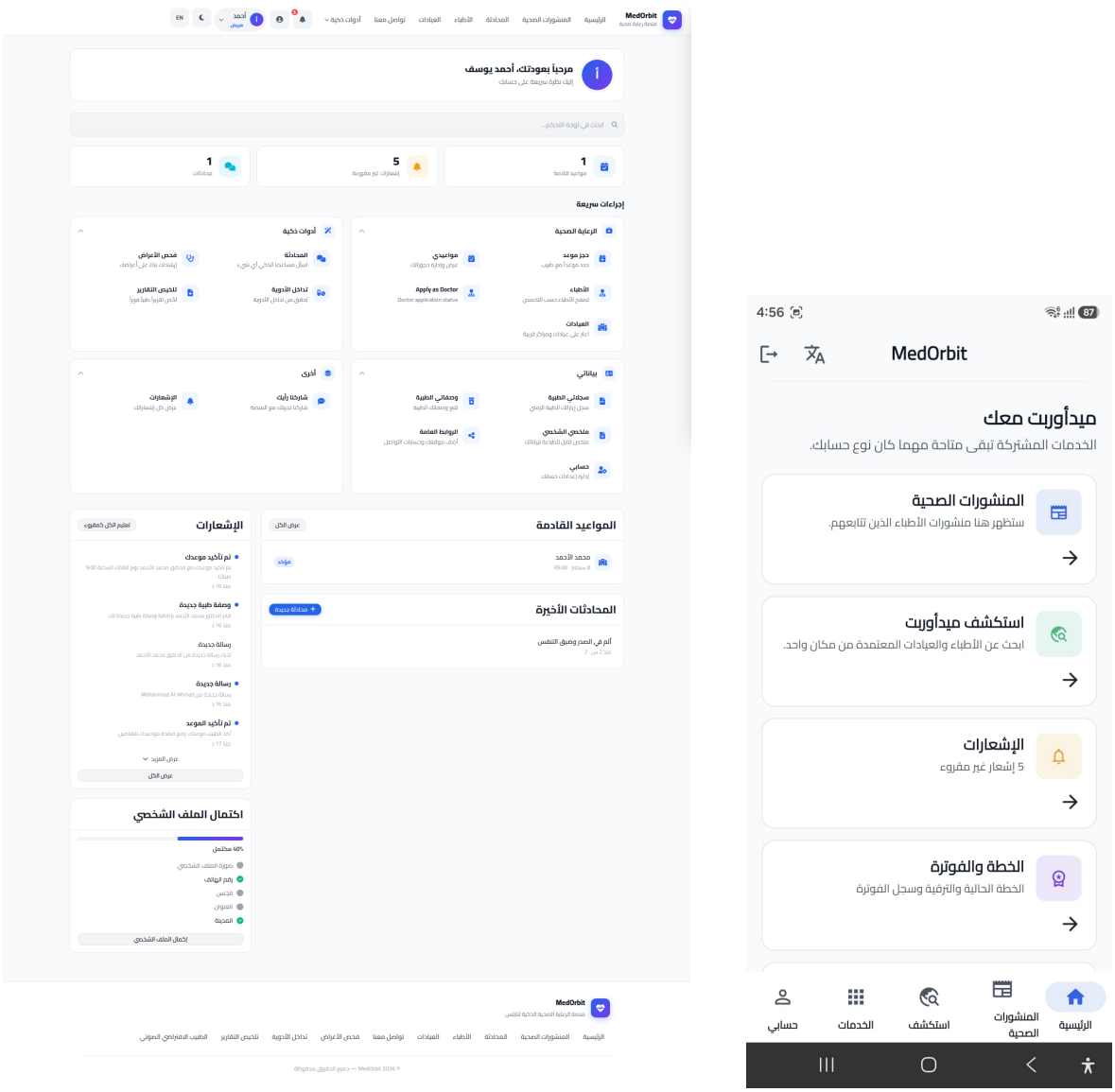
## 3.9 Web and Mobile Interface Screenshots

The figures below document eleven representative pages from the final integrated web build, each paired with the corresponding Flutter mobile screen for the same feature, captured from a real device and placed side by side so the two clients can be compared directly. These pages were chosen to cover the platform's main areas – authentication, core patient workflows, AI assistance, subscription management, and administration – rather than reproducing every implemented page. The web screenshots were captured from the fully seeded, live local deployment described in Chapter 4, and the mobile screenshots were captured from the Flutter client running against the same seeded backend, so both sides show the same underlying demo data.




(a) Web (b) Mobile

Figure 3.3: Login — web and mobile.

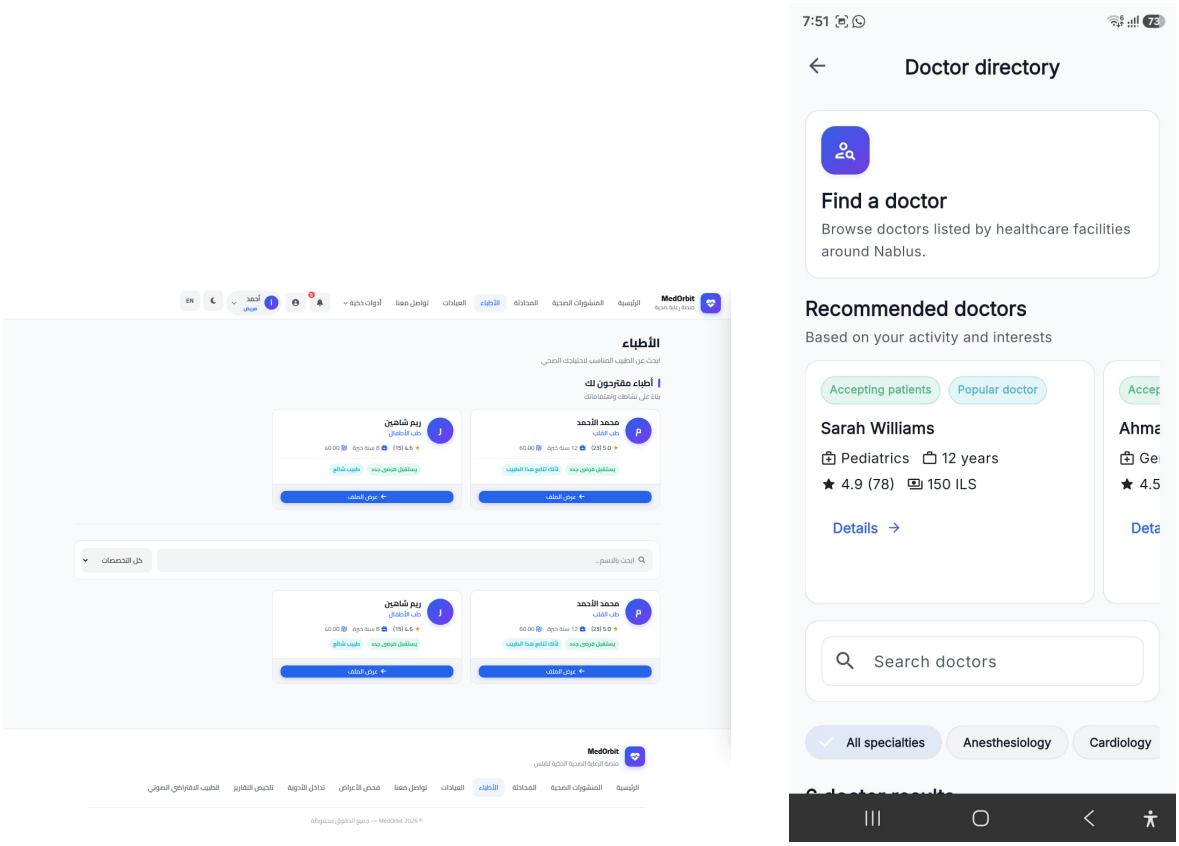



(a) Web



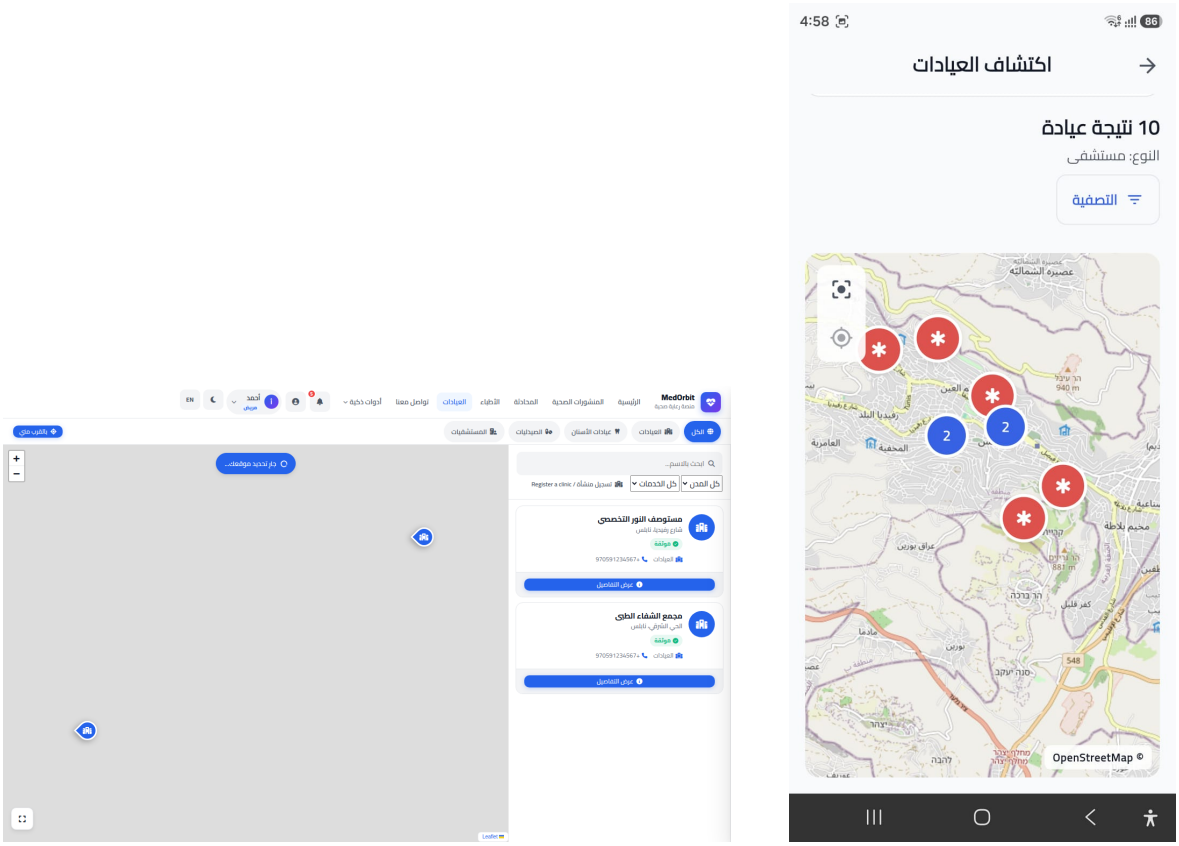

(b) Mobile

Figure 3.4: Patient dashboard — web and mobile.



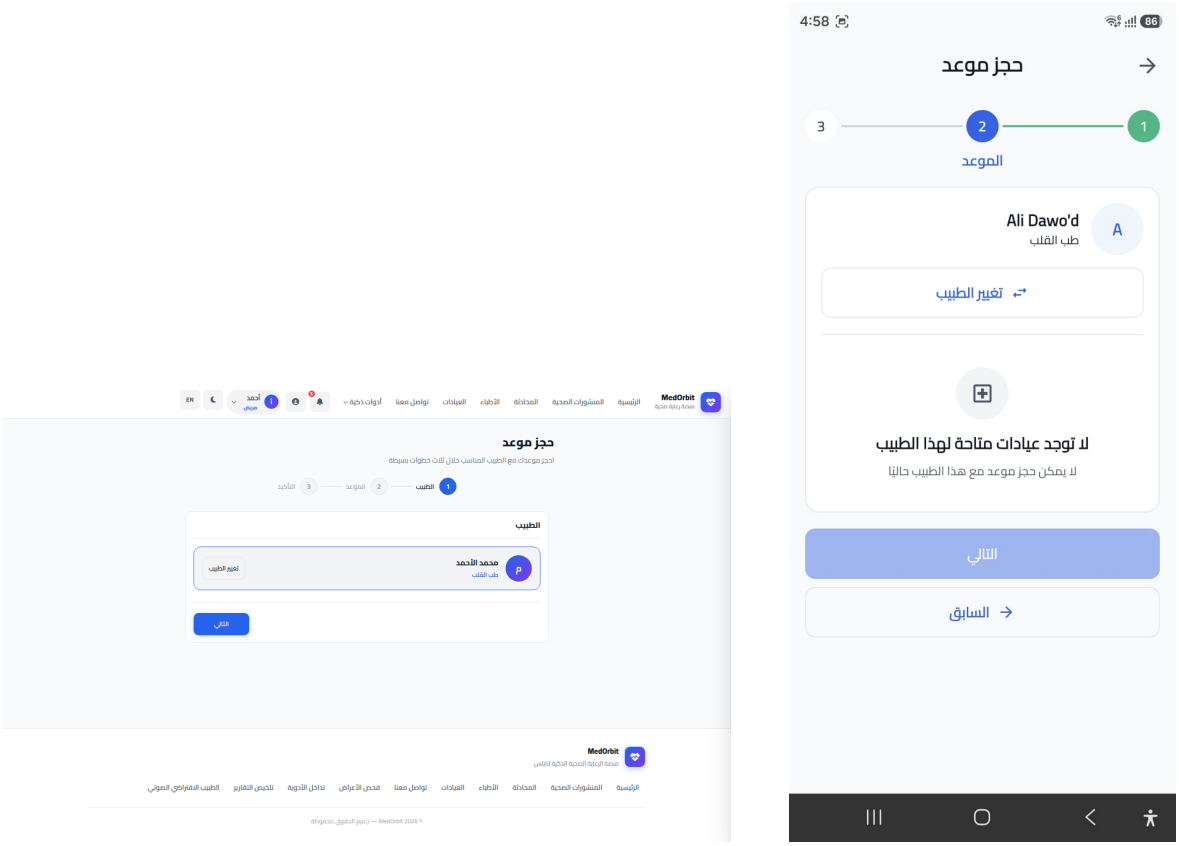

(a) Web (b) Mobile

Figure 3.5: Doctor discovery — web and mobile.



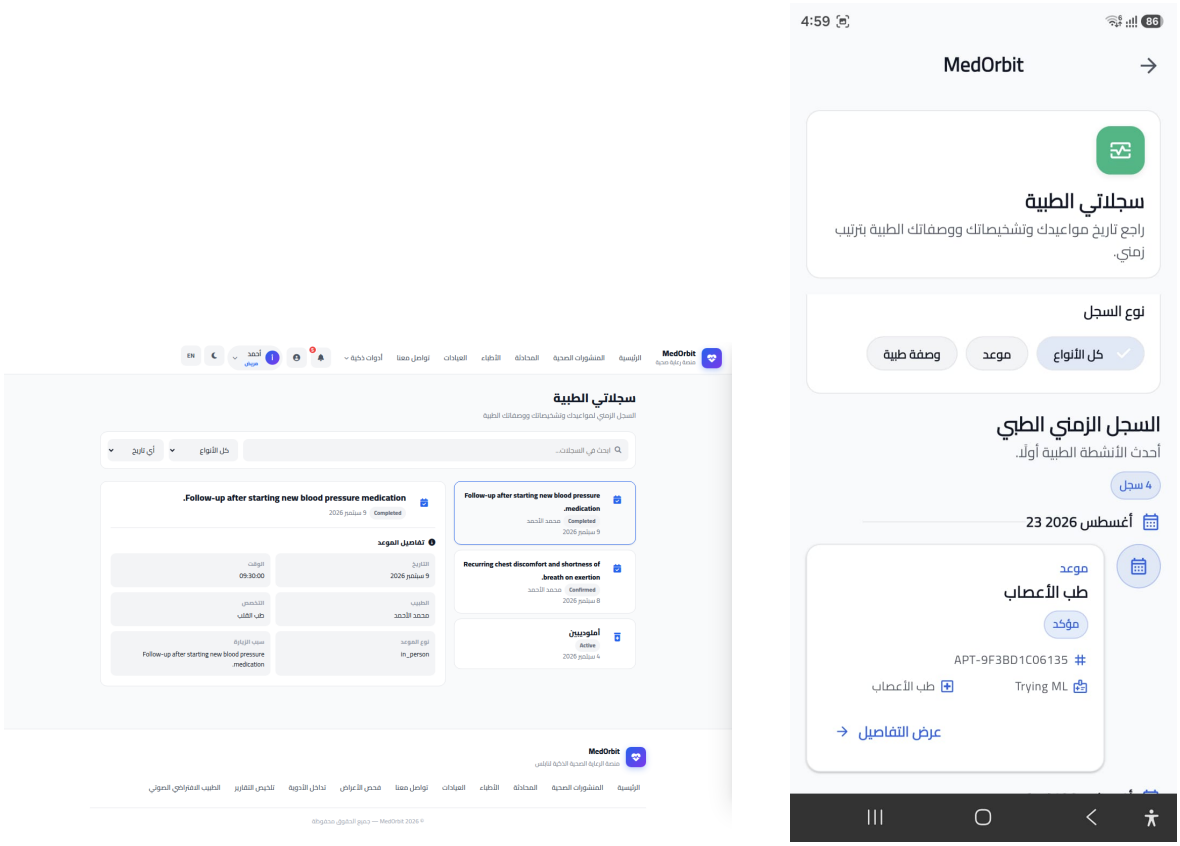

(a) Web (b) Mobile

Figure 3.6: Clinic map — web and mobile.



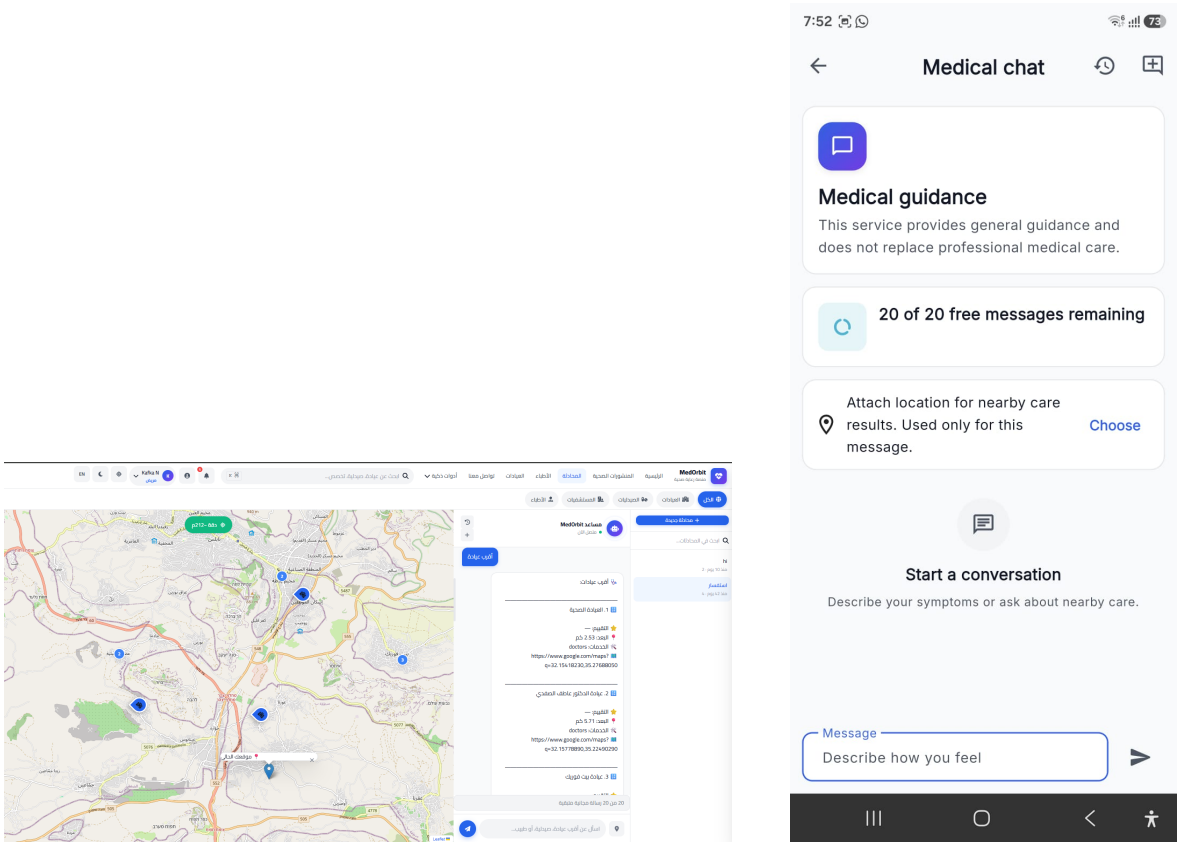

(a) Web (b) Mobile

Figure 3.7: Book appointment — web and mobile.



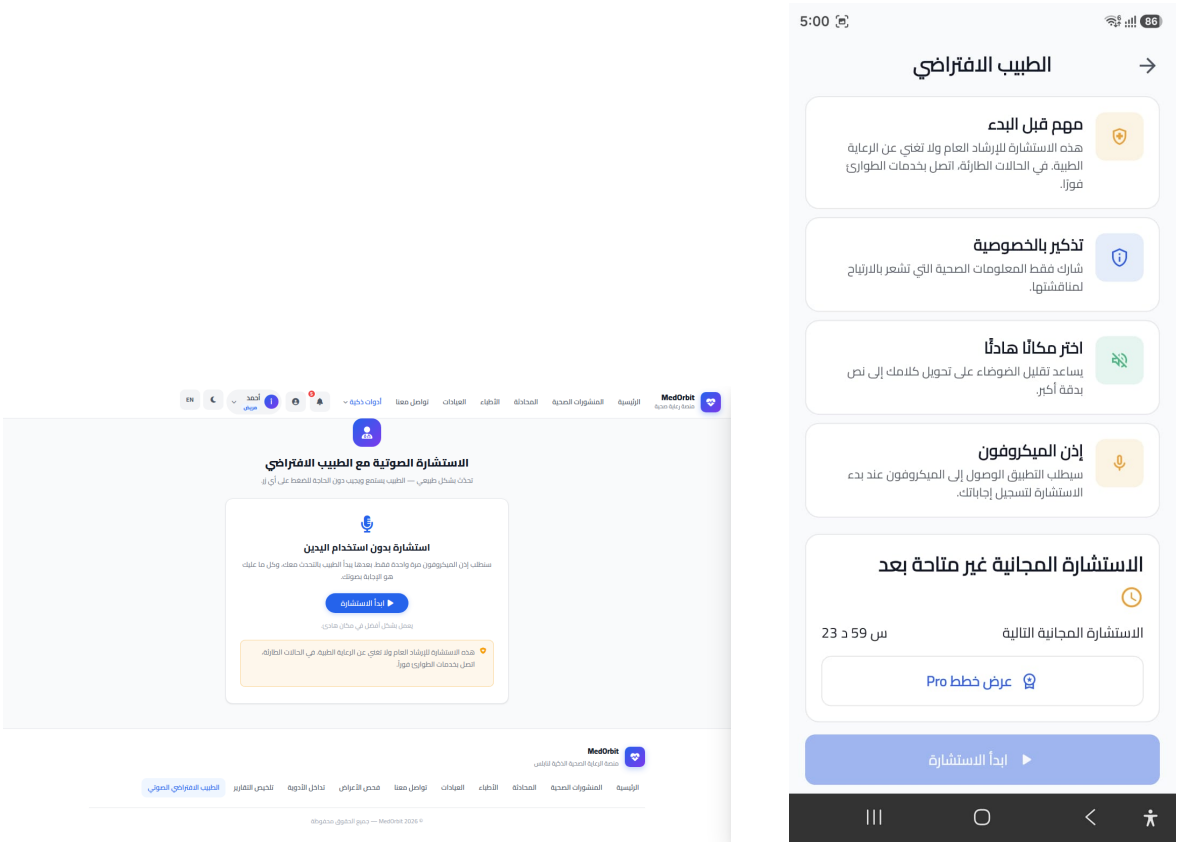

(a) Web (b) Mobile

Figure 3.8: Medical records — web and mobile.



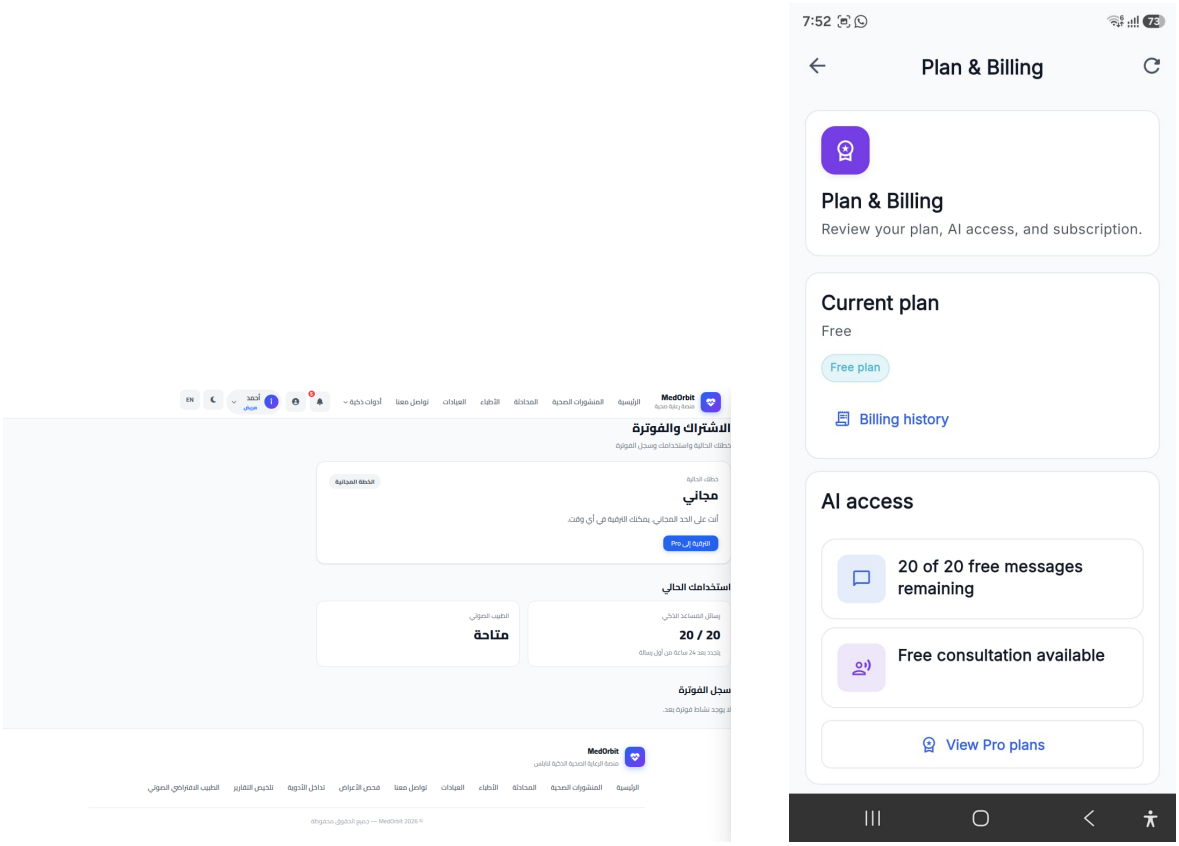

(a) Web (b) Mobile

Figure 3.9: AI assistant chat — web and mobile.



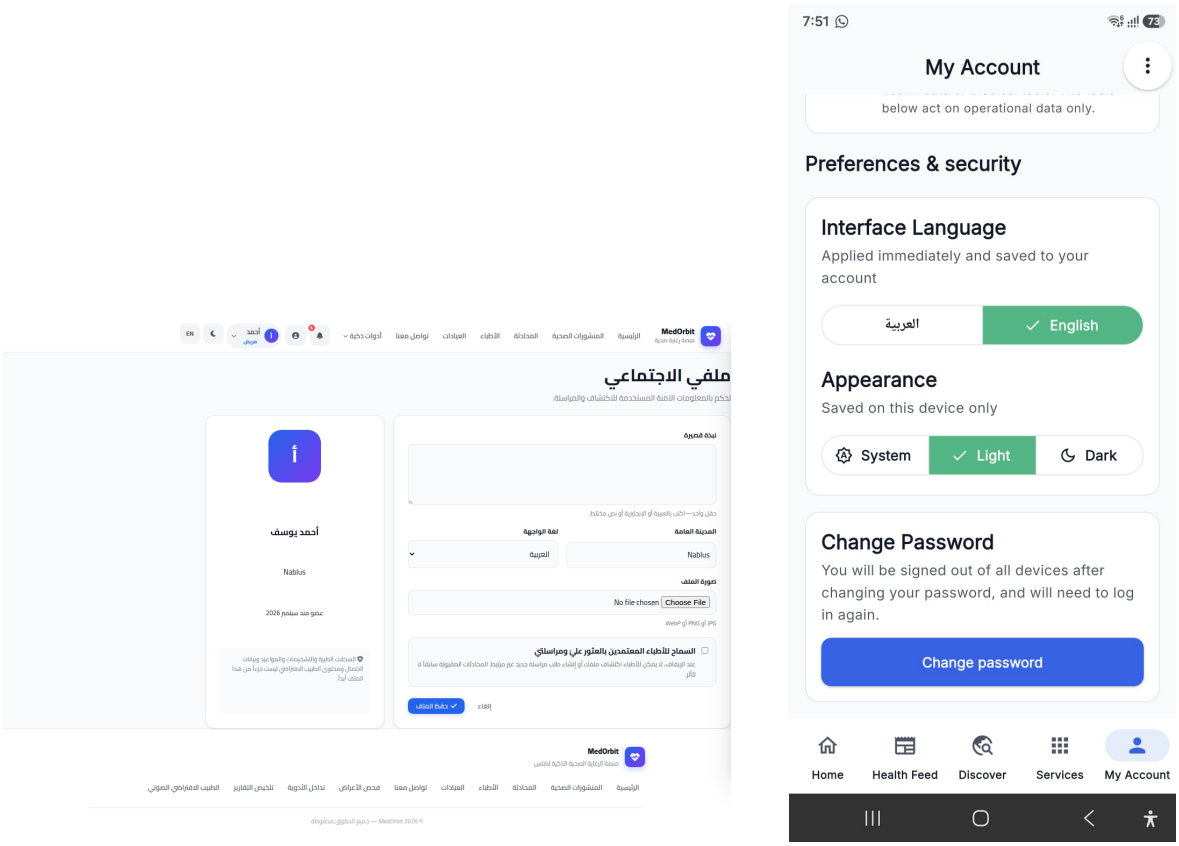

(a) Web (b) Mobile

Figure 3.10: Virtual Doctor — web and mobile.



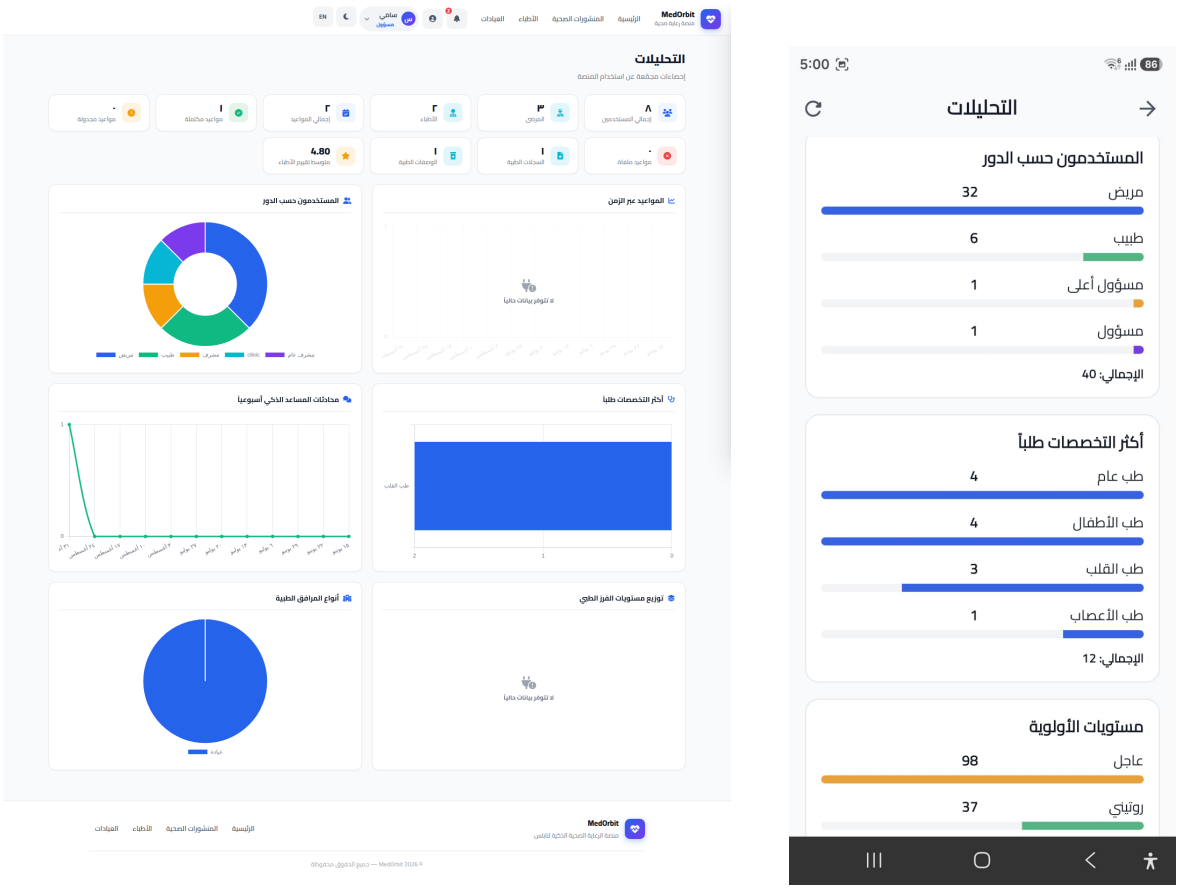

(a) Web (b) Mobile

Figure 3.11: Plan and billing — web and mobile.




(a) Web (b) Mobile

Figure 3.12: Patient profile — web and mobile.




(a) Web (b) Mobile

Figure 3.13: Admin analytics — web and mobile.

## 3.10 Appointment and Scheduling Workflow

Appointments connect patients and doctors through a common data model. Patients can browse doctors and initiate booking, while doctors manage their availability and view doctor-specific appointment information. The backend validates appointment requests and protects doctor-only scheduling operations.

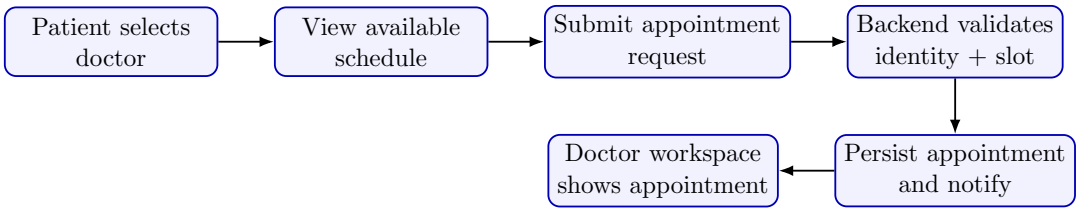



Figure 3.14: Simplified appointment scheduling workflow.

## 3.11 Medical Records and Prescriptions

The medical-record and prescription features are treated as clinical data, not general social content. Backend routes separate medical records and prescriptions from user-profile endpoints. Doctor-authored information is made available to the appropriate patient according to the application's authorization rules and care relationship. Prescription workflows can interact with the AI service's prescription-check logic. When medication data is sent for checking, the AI service uses database-backed known-interaction data and returns warnings. The warning supports the doctor but does not replace clinical judgment.

## 3.12 Direct Messaging and Notifications

MedOrbit distinguishes direct patient–doctor messaging from AI chatbot conversations. Human conversations are exposed through dedicated message routes, while AI conversation history is stored through separate chatbot conversation routes. This distinction helps keep two very different communication contexts from being mixed. The backend includes Socket.IO support for real-time behavior and notification endpoints for asynchronous user feedback. Email processing is also implemented as a background worker for queued email tasks.

## 3.13 Social Feed and Recommendations

The platform includes social content and doctor-related posting features. The recommendation subsystem is not embedded directly inside the page rendering logic. Instead, the backend includes event foundations, transactional outbox processing, Kafka, user-interest/recommendation components, and a dedicated recommendation consumer. This architecture makes it possible to evolve recommendation logic without coupling every recommendation calculation to a synchronous web request.

## 3.14 Artificial Intelligence Service

The AI subsystem is implemented as a separate FastAPI service. It connects to the database using asynchronous PostgreSQL access and exposes internal operations for several medical-assistance features.

### 3.14.1 Medical Chatbot

The chatbot pipeline includes intent classification, entity extraction, NLU processing, slot filling, provider resolution, medical safety handling, retrieval, and LLM response generation. The backend applies authentication and rate-limiting before the public client reaches persisted chatbot operations.

### 3.14.2 Symptom Triage

The triage endpoint accepts one or more symptoms and produces a triage level, recommended specialty, confidence score, matched keywords, recommendations, and follow-up action. The result is persisted in a triage-session table. The feature is deliberately framed as specialty guidance rather than a final diagnosis.

### 3.14.3 Drug Interaction Checker

The drug checker accepts medication identifiers or medication names. Interaction records are obtained from database-backed medication data, and the response includes the detected interactions and severity summary. A separate prescription-check operation can evaluate prescription items and return warning messages.

### 3.14.4 Medical Report Summarization

The report-summarization pipeline supports raw text or uploaded PDF/image input. Files pass through extraction/OCR where necessary. The resulting text is summarized into Arabic and English and is stored with processing metadata. This allows the user to review a simplified version while retaining the original document as the primary source.

### 3.14.5 Virtual Doctor

The Virtual Doctor module is a larger structured subsystem rather than one chat prompt. The repository includes an interview engine, memory, planning/reasoning components, retrieval, report generation, response handling, and voice-related components. Speech-to-text support is based on faster-whisper and text-to-speech support is based on Piper. The deployment configuration preserves model/voice caches so repeated container rebuilds do not repeatedly download large assets.

### 3.14.6 AI Processing Architecture

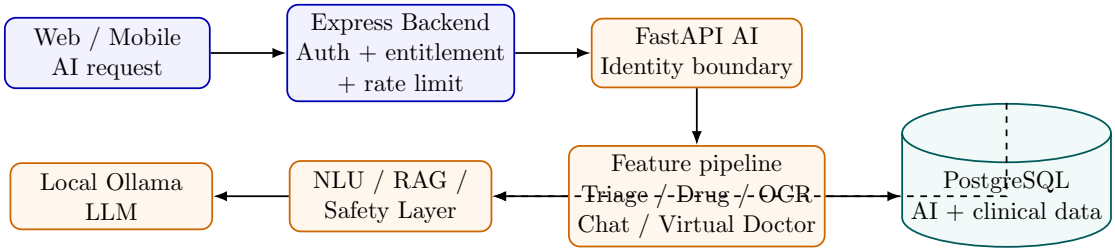



Figure 3.15: Simplified MedOrbit AI request pipeline.

## 3.15 Event-Driven Processing

The Docker stack runs three Node.js background workers with the same backend image:

- **Outbox worker**: publishes durable application events.
- **Event consumer**: processes event-side effects.
- **Recommendation consumer**: updates recommendation-related state.

Kafka is configured as the broker. Separate health ports and health checks allow Docker Compose to verify that each worker is operational. This design separates asynchronous work from the main API process.

## 3.16 Deployment and Containerization

The local Docker Compose stack includes PostgreSQL/PostGIS, Kafka, backend, AI service, frontend, and the background workers. The AI service reaches Ollama on the host through `host.docker.internal`; the LLM itself is not rebuilt as a project container. The repository also defines test-profile services for disposable integration testing and tool-profile services for database bootstrap/migrations. Staging-specific configuration is documented separately for Azure VM deployment.

## 3.17 Testing Method

Testing is divided by layer and by system boundary. The repository includes backend integration tests, frontend contract/UI tests, Python AI tests, and Flutter tests. GitHub Actions defines automated gates that build containers, initialize a fresh database, apply migrations, execute backend integration tests, start the application stack, and verify backend/AI health. A separate AI job installs the OCR and symbolic-reasoning runtime dependencies and runs AI unit, characterization, symbolic, and safety tests. The main categories are:

- Authentication and account-verification testing.
- Authorization and clinical-access testing.
- Administrator-foundation and doctor-lifecycle testing.
- Care-relationship, social-feed, and direct-messaging testing.
- Event, outbox, recommendation, and ML-readiness testing.
- Billing-entitlement and checkout-lifecycle testing.
- Frontend API-origin and targeted interface tests.
- AI unit, safety, entitlement-boundary, and characterization tests.
- Flutter widget/unit tests and static analysis.
- Manual browser and physical-device smoke testing for complete user flows.

# Chapter 4

# Results and Analysis

## 4.1 Final Integrated Platform

The final codebase contains an integrated web client, Flutter mobile client, Express backend, PostgreSQL/PostGIS database, FastAPI AI service, Kafka broker, and background workers. The architecture is therefore no longer a single demonstration page or isolated prototype. The major project domains share authentication, data, API contracts, and deployment configuration. The integration result is especially important because the original project objective required a unified digital healthcare environment rather than independent examples. The same backend supports the two client platforms, and the AI service is reached through backend-controlled boundaries rather than by exposing the internal AI API directly to the user interface.

## 4.2 Authentication and Role Management Result

The system provides registration, login, verification, password recovery, and Google authentication flows. Role-aware behavior separates patient, doctor, administrator, and super-administrator capabilities. Doctor application and administrator invitation workflows provide controlled transitions into privileged roles instead of allowing a user to select a privileged role directly. This structure improves the integrity of the role model. The application can present different navigation for different users while the backend remains the authoritative location for access decisions.

## 4.3 Doctor and Clinic Discovery Result

Doctor and clinic/facility discovery were implemented as first-class features. Doctor pages expose specialty-oriented discovery and doctor detail, while clinic data can be used in a geographic context through PostGIS and map libraries. Figures 3.5 and 3.6

document the implemented list and map-based discovery views.

## 4.4 Appointment Result

Appointment routes exist in both web and mobile flows, and the backend has a dedicated appointment route group. The mobile client also includes doctor workspace routes for doctor appointments and schedule management. This demonstrates that scheduling is implemented from both sides of the patient–doctor relationship rather than as a patient-only form.

## 4.5 Medical Records and Prescription Result

Medical records and prescriptions are exposed through dedicated backend route groups and mobile routes. The separation of records from general user content allows these resources to receive different authorization rules. Prescription data can also be checked through the AI service's interaction-checking logic. Figure 3.8 shows the web and mobile presentation of the medical-records screen.

## 4.6 Communication Result

MedOrbit provides notification infrastructure, direct messages between human users, AI conversations, and contact/feedback flows. The backend's explicit separation between `/api/messages` and AI chatbot conversation routes is an important result because it avoids treating a medical chatbot as if it were a human healthcare professional.

## 4.7 AI Feature Results

The implemented FastAPI service exposes structured operations for symptom triage, drug interactions, prescription checking, medical-report summarization, chatbot processing, and Virtual Doctor functions.

Table 4.1: Implemented AI feature outputs.

| Feature | Input | Main Output |
|---|---|---|
| Medical chatbot | Natural-language question/context | Intent-aware response with retrieved context and safety processing |
| Symptom triage | List of reported symptoms | Triage level, specialty recommendation, confidence, follow-up action |
| Drug checker | Medication IDs or names | Interaction list and severity summary |
| Prescription check | Prescription medication items | Safety flag, interaction data, warnings |
| Report summarizer | Text, PDF, or image | Extracted text plus Arabic and English summaries |
| Virtual Doctor | Structured interview/voice/text interaction | Guided interview state, reasoning/response and generated report-oriented output |

Figures 3.9 and 3.10 in Section 3.9 show the AI assistant chat and the Virtual Doctor voice-consultation feature as representative examples, with the web presentation of each paired with the corresponding mobile screen.

## 4.8 Administration Result

The platform includes administrative pages and routes for user management, doctor-application review, analytics, contact messages, social moderation, audit logs, and invitations. The mobile route map also exposes dedicated administration screens. This shows that platform governance was treated as a separate role rather than implemented as unrestricted database access.

## 4.9 Billing and Entitlement Result

Billing routes, subscription/history screens, and entitlement-related testing are present in the implementation. The backend distinguishes product quota from infrastructure fair-use rate limiting for AI chat. Test configuration uses an isolated mock provider rather than enabling simulated payments in the normal production-mode backend process. This design is useful for a graduation prototype because it allows the team to test the subscription state machine without pretending that a sandbox provider is a real payment processor.

## 4.10 Event and Recommendation Result

The repository contains a transactional-outbox migration, Kafka services, background event consumers, recommendation processing, and tests for event foundations and deterministic recommendations. This demonstrates an architectural extension beyond direct request/response CRUD operations. The resulting design can react to user and system events asynchronously, which is useful for recommendation profiles and notification-like workloads.

## 4.11 Testing Summary

Table 4.2 summarizes the evidence represented in the project test and CI structure. The current web screenshots are included in this report; the final physical mobile-device evidence can be inserted after the remaining mobile capture session.

Table 4.2: Summary of project verification areas.

| **Verification Area** | **Coverage** | **Evidence in Project** |
|---|---|---|
| Authentication / verification | Automated | Backend auth and verification test suites |
| Authorization / clinical access | Automated | Authorization and clinical-boundary tests |
| Administration | Automated | Admin foundation, notification, analytics and related tests |
| Doctor lifecycle / scheduling | Automated | Doctor lifecycle and scheduling tests |
| Messaging / social | Automated | Direct-messaging and social-feed tests |
| Events / recommendations | Automated | Event, Kafka, recommendation and signal tests |
| Billing / entitlements | Automated | Entitlement and checkout-lifecycle tests |
| Frontend contracts | Automated | API-origin and targeted UI tests |
| AI service | Automated | Unit, safety, symbolic, characterization, and identity tests |
| Database startup | Automated in CI | Fresh bootstrap, ordered migrations, bootstrap verification |
| Integrated services | Automated in CI | Docker builds and backend/AI health checks |
| Mobile UI flows | Test + manual | Flutter tests and static analysis; final device screenshots pending |

## 4.12 Analysis of the Architecture

A major strength of the final implementation is separation of concerns. Web/mobile presentation, backend business rules, AI processing, data storage, and asynchronous workers are distinct components. This separation allows a failure or change in one layer to be reasoned about without rewriting the entire platform. A second strength is the shared backend contract. Web and mobile clients do not maintain separate databases or separate healthcare rules. This reduces inconsistent behavior and allows role/access fixes to be applied centrally. A third strength is the explicit treatment of AI as a subsystem. AI features are not allowed to become the authentication authority or the source of all clinical facts. Deterministic database-backed checks are used where appropriate, and the LLM is surrounded by retrieval and safety processing.

# Chapter 5

# Discussion

## 5.1 Main Achievements

The primary achievement of MedOrbit is the implementation of a broad healthcare platform as one integrated software system. The final repository contains multiple clients, a shared backend, a relational/geospatial database, dedicated AI processing, asynchronous infrastructure, testing, and containerized deployment. Another important achievement is role depth. The system is not limited to a single "user" account. Patient, doctor, administrator, and super administrator workflows are reflected in both routes and interfaces. Doctor applications, care relationships, audit-related functionality, and admin invitations show that account privileges were modeled as managed workflows. The project also progressed beyond a simple AI-chat demonstration. Triage, drug-interaction checks, report extraction/summarization, retrieval, and the Virtual Doctor module use different technical pipelines according to the problem being solved.

## 5.2 Full-Stack Integration Challenges

A large integrated platform creates contract problems. A change to an API response can affect the static web client and Flutter client at the same time. Authentication state, error formats, pagination, role values, and resource identifiers must therefore stay consistent across multiple codebases. The project addresses this with shared API-contract documentation, backend integration tests, frontend tests, and a mobile routing structure that maps features explicitly. Nevertheless, maintaining parity between web and mobile remains a continuing engineering cost.

## 5.3 Authentication and Authorization Challenges

Healthcare-oriented data makes authorization more complex than checking whether a request contains a token. A doctor may be authenticated but should not automatically access every patient's records. An administrator may manage platform data but should not be treated as the author of clinical records. A super administrator may invite administrators, while a normal administrator should not receive the same privilege. These examples required role-specific middleware and relationship-aware rules. Testing authorization is therefore as important as testing successful feature behavior.

## 5.4 AI and Medical-Safety Challenges

The largest conceptual risk in an AI-healthcare application is presenting generated language as medical certainty. A fluent model response can be wrong, incomplete, or based on weak context. OCR adds another source of error when reports are scanned, and retrieval can fail to return the most relevant passage. For this reason, MedOrbit's AI features should always display clear limitations. Emergency symptoms should direct the user toward urgent professional care, drug-checking results should not override a pharmacist or doctor, and report summaries should not replace the original report. This approach is consistent with the need for human oversight emphasized in WHO AI-for-health guidance [2, 3].

## 5.5 Database and Migration Challenges

The database is shared by many features and evolves continuously. A direct manual schema change can make a clean installation behave differently from a developer's existing database. MedOrbit therefore uses a tracked historical baseline plus ordered migrations and includes migration verification in CI. Geospatial support also requires a PostGIS-capable PostgreSQL image, while AI and recommendation features introduce additional tables and data relationships. The database design is therefore a central integration dependency rather than a passive storage layer.

## 5.6 Event-Driven Architecture Challenges

Asynchronous processing improves separation but introduces new failure modes. An event may be published late, a consumer may restart, or a message may be processed more than once. The transactional-outbox approach improves durability, while separate worker health checks improve operational visibility. For a graduation project,

this architecture adds complexity, but it also demonstrates how the system can move beyond synchronous CRUD design.

## 5.7 Mobile Development Challenges

The mobile application must operate on different screen sizes while preserving role-aware navigation and Arabic/English direction. It also has to handle API connectivity, secure local storage, device permissions, geolocation, image/file selection, audio features, and real-time messaging. Physical-device testing is especially important because emulator-only tests cannot fully represent permission dialogs, network routing, keyboard behavior, screen constraints, and audio interaction. The screenshot phase of this report will therefore be performed on the final integrated build rather than using mock images.

## 5.8 Security Considerations

The project includes practical security controls such as password hashing, JWT-based authentication, CORS configuration, security headers, request-size limits, rate limiting, controlled internal AI identity, and privileged administrative boundaries. However, a secure academic prototype should not be confused with a complete security certification. A production healthcare system would require deeper threat modeling, penetration testing, key/secret management, encryption policy review, backup and disaster-recovery procedures, privacy-impact assessment, legal/compliance review, monitoring, incident response, and secure production infrastructure.

## 5.9 Limitations

The current project has several limitations:

- It is an academic prototype and is not a certified medical device.
- AI responses can be inaccurate and must not be treated as definitive medical advice.
- Medical-report OCR quality depends on the source document and scan quality.
- Drug-interaction checking is limited by the completeness and correctness of the interaction dataset.
- The local LLM requires sufficient host resources and can produce slower responses than ordinary REST operations.
- Map/facility usefulness depends on the quality and freshness of available Nablus facility data.

- A production deployment would require stronger privacy, compliance, monitoring, backup, and operational controls.
- Billing integration used for development/testing must be connected to and reviewed against a real provider before commercial use.
- The web screenshots are included; the remaining final mobile screenshots and device-level acceptance evidence will be inserted after the final mobile capture session.

## 5.10 Comparison with Project Objectives

Table 5.1: Comparison between project objectives and the implemented system.

| **Objective** | **Result** | **Implementation Evidence** |
|---|---|---|
| Unified web and mobile platform | Achieved | Static web client and Flutter client use shared backend contracts |
| Secure user and role management | Achieved at prototype level | Auth routes, JWT/password security, role/admin boundaries |
| Doctor/clinic discovery | Achieved | Doctor, specialty and clinic routes; PostGIS/map clients |
| Appointment scheduling | Achieved | Patient booking and doctor scheduling/appointment surfaces |
| Digital records/prescriptions | Achieved | Dedicated clinical route groups and mobile/web surfaces |
| Patient–doctor communication | Achieved | Direct messaging, care relationships and notifications |
| AI medical assistance | Achieved as decision support | Chatbot, triage, drug checker, report summarizer, Virtual Doctor |
| Arabic/English support | Achieved | Shared localization and RTL-aware clients |
| Event-driven recommendations | Achieved | Outbox, Kafka, event/recommendation workers |
| Containerized integration | Achieved | Docker Compose stack and staging configuration |
| Production medical certification | Out of scope | Requires regulatory and operational processes beyond graduation prototype |

# Chapter 6

# Conclusions and Recommendations

## 6.1 Conclusion

MedOrbit successfully demonstrates the design and implementation of an integrated smart-healthcare software platform. The project combines a web application, Flutter mobile application, Node.js/Express backend, PostgreSQL/PostGIS database, Python/FastAPI AI service, local LLM integration, real-time communication, event-driven processing, and containerized deployment. The project achieved its central objective of bringing multiple healthcare workflows into one environment. Patients can discover care, arrange appointments, communicate, review clinical information, and access AI-assisted tools. Doctors receive dedicated scheduling, patient, record, communication, and professional-profile functions. Administrators receive separate management and review functions. From an engineering perspective, the most important result is not the number of pages or features but the integration boundaries between them. Shared API contracts, database migrations, role-aware authorization, an internal AI identity boundary, asynchronous workers, and automated tests make the project more maintainable than a collection of unrelated demonstrations. The AI subsystem expands the project while preserving an important principle: AI is an assistance layer and not an autonomous medical authority. Structured triage, database-backed drug checks, OCR-aware report summarization, RAG, and Virtual Doctor workflows demonstrate several ways AI can be integrated while still requiring professional medical judgment.

## 6.2 Recommendations and Future Work

Several improvements can be considered in future versions:

- Conduct formal usability studies with patients and healthcare professionals.
- Add production-grade monitoring, centralized logging, alerting, backup, and dis-

aster recovery.

- Perform a formal privacy/security threat model and independent penetration testing.
- Review the complete platform against applicable Palestinian healthcare/privacy requirements and any target-market regulations before real deployment.
- Expand and clinically review the drug-interaction dataset.
- Evaluate symptom-triage accuracy against a labeled clinical dataset under professional supervision.
- Add stronger source citation and confidence presentation to AI-generated medical answers.
- Improve OCR quality assessment and show low-confidence extraction warnings.
- Introduce robust production payment integration only after provider and compliance review.
- Expand clinic/facility data beyond Nablus while maintaining data quality and location accuracy.
- Add production-grade video consultation if required by the final deployment scope.
- Strengthen mobile accessibility testing and expand testing to additional Android/iOS devices.
- Collect performance measurements for API latency, AI response time, database queries, and concurrent users.
- Continue improving recommendation quality using privacy-aware signals and transparent controls.

# Bibliography


[1] World Health Organization. *Global Strategy on Digital Health 2020–2027*. WHO, updated 2025. Available at: https://www.who.int/publications/i/item/9789240116870.

[2] World Health Organization. *Ethics and Governance of Artificial Intelligence for Health: WHO Guidance*. WHO, 2021. Available at: https://www.who.int/publications/i/item/9789240029200.

[3] World Health Organization. *Ethics and Governance of Artificial Intelligence for Health: Guidance on Large Multi-Modal Models*. WHO, 2025. Available at: https://www.who.int/publications/i/item/9789240084759.

[4] Google. *Flutter Documentation*. Available at: https://docs.flutter.dev/.

[5] OpenJS Foundation. *Node.js Documentation*. Available at: https://nodejs.org/docs/latest/api/.

[6] OpenJS Foundation. *Express Documentation*. Available at: https://expressjs.com/.

[7] PostgreSQL Global Development Group. *PostgreSQL Documentation*. Available at: https://www.postgresql.org/docs/.

[8] PostGIS Project. *PostGIS Documentation*. Available at: https://postgis.net/documentation/.

[9] FastAPI. *FastAPI Documentation*. Available at: https://fastapi.tiangolo.com/.

[10] Apache Software Foundation. *Apache Kafka Documentation*. Available at: https://kafka.apache.org/documentation/.

[11] Docker Inc. *Docker Compose Documentation*. Available at: https://docs.docker.com/compose/.

[12] Leaflet. *Leaflet Documentation*. Available at: https://leafletjs.com/reference.html.

[13] Ollama. *Ollama Documentation*. Available at: https://docs.ollama.com/.

# Appendix A

# Main API Functional Groups

| API Group | Purpose |
|---|---|
| `/api/auth` | Login, registration, verification and authentication flows |
| `/api/users` | User profile/account operations |
| `/api/doctors` | Doctor discovery, details and doctor-related operations |
| `/api/clinics` | Clinic/facility discovery and location-related operations |
| `/api/specialties` | Medical specialty data |
| `/api/appointments` | Appointment booking and management |
| `/api/medical-records` | Electronic medical records |
| `/api/prescriptions` | Electronic prescriptions |
| `/api/messages` | Human direct messaging |
| `/api/conversations` | AI/chat conversation persistence |
| `/api/chat` | Medical chatbot requests |
| `/api/ai` | Authenticated AI feature bridge |
| `/api/virtual-doctor` | Virtual Doctor backend boundary |
| `/api/notifications` | User notification operations |
| `/api/feed` | Social feed |
| `/api/recommendations` | Recommendation retrieval |
| `/api/billing` | Subscription/billing/entitlement operations |
| `/api/doctor-applications` | Doctor application submission/status |
| `/api/admin/...` | Administration, applications, moderation, invitations, settings, audit and contact management |

# Appendix B

# Mobile Route Summary

| Area | Representative Routes |
|---|---|
| Authentication | Login, register, forgot password, verify code, reset password |
| Core navigation | Home, feed, discover, services, profile |
| Clinical data | Records, prescriptions, reports |
| Scheduling | Appointments, appointment booking |
| AI tools | Symptom checker, drug checker, report summarizer, chatbot, Virtual Doctor |
| Discovery | Clinics, clinic detail, doctors, doctor detail, map |
| Care network | My doctors, doctor notes, my doctor |
| Communication | Messages, new message, thread, notifications, contact |
| Billing | Billing, subscription, history, sandbox |
| Doctor workspace | Workspace, professional profile, schedule, appointments, patients, posts, records |
| Administration | Dashboard, analytics, users, doctor applications, contact messages, moderation, audit logs, invitations |

# Appendix C

# Screenshot Checklist

Eleven representative pages were selected for this report rather than every implemented page, covering authentication, core patient workflows, AI assistance, subscription management, and administration. Both the web and the mobile screenshot for every entry below are real captures: the web screenshots were captured from the fully seeded, live local deployment described in Chapter 4, and the mobile screenshots were captured from the Flutter client running on a real device against that same seeded backend, so both sides show the same underlying demo data. Both sets are included in the report (under `screenshots/web/` and `screenshots/mobile/` respectively) and appear together in the paired figures of Section 3.9.

## Web and Mobile Screenshots — Completed

1. Login — `02-login.png`
2. Patient dashboard — `05-patient-dashboard.png`
3. Doctor discovery — `06-find-doctors.png`
4. Clinic map — `07-find-clinics.png`
5. Book appointment — `09-book-appointment.png`
6. Medical records — `12-my-records.png`
7. AI assistant chat — `04-chatbot-home.png`
8. Virtual Doctor — `19-virtual-doctor.png`
9. Plan and billing — `24-billing.png`
10. Patient profile — `21-patient-profile.png`
11. Admin analytics — `34-admin-analytics.png`

# Appendix D

# Final Acceptance Evidence

The web and mobile capture sets are both complete, including real physical-device mobile screenshots and Arabic/English (RTL) examples across the pairs in Section 3.9. Before final submission, this appendix should additionally contain:

- final Docker service-health screenshot or exported verification;
- final web role smoke-test evidence;
- representative AI feature outputs with medical disclaimer visible;
- final database/ER diagram;
- any measured performance data that the team chooses to report.

Only measured values from actual testing should be inserted. No response time, accuracy percentage, concurrent-user capacity, or clinical-performance claim should be invented.